Practical assignment 1

Field	Detail
Module	**OOP**
NAME	RONALD MURAPE
DEPARTMENT	COMPUTER SCIENCE
REG NUMBER	H250475V
COURSE CODE	I

task 1 Abstract class Account

Account is declared abstract, so it cannot be instantiated such that there is no such thing as a plain account at the bank and everything every account shares is written once:

- accountNumber and balance are protected, so subclasses can read and update them but outside code cannot. An extra field, accountHolder, stores the customer name.
- The constructor initialises the fields and rejects a blank account number or a negative opening balance.
- deposit() is concrete because deposits work the same for every account. It rejects amounts that are zero, negative, NaN or infinite.
- withdraw() and endOfMonth() are abstract: the base class declares that every account can do these things, and each subclass decides how. endOfMonth() is a hook the bank calls at month-end.

Account.java code

```
/**
 * Base type for every bank account. Holds the behaviour all accounts share
 * (deposits, balance tracking) and forces each subclass to decide its own
 * withdrawal and month-end rules.
 */
public abstract class Account {

    protected final String accountNumber;
    protected final String accountHolder;
    protected double balance;

    protected Account(String accountNumber, String accountHolder, double openingBalance) {
        // Fail fast: an account with no number or a negative opening balance is a data error,
not a user mistake
        if (accountNumber == null || accountNumber.isBlank()) {
            throw new IllegalArgumentException("Account number must not be empty.");
        }
        if (accountHolder == null || accountHolder.isBlank()) {
```

```
            throw new IllegalArgumentException("Account holder name must not be empty.");
        }
        if (openingBalance < 0) {
            throw new IllegalArgumentException("Opening balance cannot be negative.");
        }
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = openingBalance;
    }

    /** Deposits work identically for every account type, so they live here once. */
    public void deposit(double amount) {
        if (!isValidAmount(amount)) {
            System.out.printf("  REJECTED: deposit of %s into %s - amount must be greater than
zero.%n",
                    formatUsd(amount), accountNumber);
            return;
        }
        balance += amount;
        System.out.printf("  Deposited %s into %s.%n", formatUsd(amount), accountNumber);
    }

    public double getBalance() {
        return balance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getAccountHolder() {
        return accountHolder;
    }

    /**
     * Each account type has its own limit on how far the balance may fall.
     *
     * @return true if the withdrawal was applied, false if it was rejected
     */
    public abstract boolean withdraw(double amount);

    /** Hook called by the bank at month-end; subclasses decide what "month-end" means for
them. */
    public abstract void endOfMonth();

    /** Lets callers label an account without needing instanceof. */
    public abstract String getAccountType();

    /** Shared so every subclass validates amounts the same way. */
    protected static boolean isValidAmount(double amount) {
        // NaN and infinity would silently corrupt the balance, so treat them as invalid too
        return amount > 0 && Double.isFinite(amount);
    }

    /** Puts the minus sign before the currency symbol so overdrawn balances read naturally.
*/
    public static String formatUsd(double amount) {
        String formatted = String.format("$%,.2f", Math.abs(amount));
        return amount < 0 ? "-" + formatted : formatted;
    }
}
```

Task 2 – SavingsAccount

The minimum balance ($50.00) and the interest rate (1% per month) are private static final constants, so they are fixed and cannot be changed from outside.

- withdraw() calculates the balance the withdrawal would leave. If that is below the minimum, it prints a rejected message and leaves the balance unchanged.
- endOfMonth() adds interest, rounded to whole cents. Both methods carry @Override, so the compiler checks that they really override the abstract methods.

SavingsAccount.java code

```
/**
 * A savings account must always keep a minimum balance and earns interest
 * at month-end.
 */
public class SavingsAccount extends Account {

    private static final double MINIMUM_BALANCE = 50.00;
    private static final double MONTHLY_INTEREST_RATE = 0.01; // 1% per month

    public SavingsAccount(String accountNumber, String accountHolder, double openingBalance) {
        super(accountNumber, accountHolder, openingBalance);
        // Opening below the minimum would put the account in a state withdraw() is designed
to prevent
        if (openingBalance < MINIMUM_BALANCE) {
            throw new IllegalArgumentException("Savings accounts must open with at least "
                    + formatUsd(MINIMUM_BALANCE) + ".");
        }
    }

    @Override
    public boolean withdraw(double amount) {
        if (!isValidAmount(amount)) {
            System.out.printf("  REJECTED: withdrawal of %s from %s - amount must be greater
than zero.%n",
                    formatUsd(amount), accountNumber);
            return false;
        }
        double balanceAfter = balance - amount;
        if (balanceAfter < MINIMUM_BALANCE) {
            System.out.printf("  REJECTED: withdrawing %s from %s would leave %s, below the
minimum balance of %s.%n",
                    formatUsd(amount), accountNumber, formatUsd(balanceAfter),
formatUsd(MINIMUM_BALANCE));
            return false;
        }
        balance = balanceAfter;
        System.out.printf("  Withdrew %s from %s.%n", formatUsd(amount), accountNumber);
        return true;
    }

    @Override
    public void endOfMonth() {
        // Round to whole cents so the stored balance always matches what the customer sees on
a statement
        double interest = Math.round(balance * MONTHLY_INTEREST_RATE * 100) / 100.0;
        balance += interest;
        System.out.printf("  Month-end: interest of %s credited at %.1f%%.%n",
                formatUsd(interest), MONTHLY_INTEREST_RATE * 100);
    }
```

```
    @Override
    public String getAccountType() {
        return "Savings";
    }
```

Task 3 – CurrentAccount

- overdraftLimit is a private final instance field because the bank agrees a limit with each customer individually.
- The constructor checks that it lies between $0 and $1,000.
- withdraw() lets the balance go negative but rejects any withdrawal that would leave it below overdraftLimit.
- When an approved withdrawal leaves the account overdrawn, the message says by how much.
- endOfMonth() deducts a flat $2.50 maintenance fee and pays no interest.

CurrentAccount.java code

```
/**
 * A current (cheque) account may go overdrawn up to an agreed limit and pays
 * a flat maintenance fee instead of earning interest.
 */
public class CurrentAccount extends Account {

    private static final double MONTHLY_MAINTENANCE_FEE = 2.50;
    private static final double MAX_OVERDRAFT_LIMIT = 1000.00;

    // Per-account because the bank agrees an overdraft with each customer individually
    private final double overdraftLimit;

    public CurrentAccount(String accountNumber, String accountHolder, double openingBalance,
double overdraftLimit) {
        super(accountNumber, accountHolder, openingBalance);
        if (overdraftLimit < 0 || overdraftLimit > MAX_OVERDRAFT_LIMIT) {
            throw new IllegalArgumentException("Overdraft limit must be between $0.00 and "
                    + formatUsd(MAX_OVERDRAFT_LIMIT) + ".");
        }
        this.overdraftLimit = overdraftLimit;
    }

    @Override
    public boolean withdraw(double amount) {
        if (!isValidAmount(amount)) {
            System.out.printf("  REJECTED: withdrawal of %s from %s - amount must be greater
than zero.%n",
                    formatUsd(amount), accountNumber);
            return false;
        }
        double balanceAfter = balance - amount;
        // The overdraft limit is a floor of -overdraftLimit, not a cap on a single withdrawal
        if (balanceAfter < -overdraftLimit) {
            System.out.printf("  REJECTED: withdrawing %s from %s would leave %s, beyond the
overdraft limit of %s.%n",
                    formatUsd(amount), accountNumber, formatUsd(balanceAfter),
formatUsd(overdraftLimit));
            return false;
        }
        balance = balanceAfter;
        if (balance < 0) {
```

```
            System.out.printf("  Withdrew %s from %s - account is now overdrawn by %s
(limit %s).%n",
                    formatUsd(amount), accountNumber, formatUsd(-balance),
formatUsd(overdraftLimit));
        } else {
            System.out.printf("  Withdrew %s from %s.%n", formatUsd(amount), accountNumber);
        }
        return true;
    }

    @Override
    public void endOfMonth() {
        // The fee is a bank charge, not a customer withdrawal, so it is applied even when
overdrawn
        balance -= MONTHLY_MAINTENANCE_FEE;
        System.out.printf("  Month-end: maintenance fee of %s deducted (no interest on current
accounts).%n",
                formatUsd(MONTHLY_MAINTENANCE_FEE));
    }

    @Override
    public String getAccountType() {
        return "Current";
    }
}
```

Task 4 – BankDemo (polymorphic loop)

- The list is declared as `List<Account>` and holds two SavingsAccount and two CurrentAccount objects.
- The loop variable has type `Account` and the code never casts or uses instanceof.
- Every account receives the same $250 withdrawal request, so any difference in the result comes from the object's own overridden method.
- The program prints the balance before and after each operation.

BankDemo.java code

```
import java.util.List;

/**
 * Runs the same operations over a mixed list of accounts to show that each
 * object applies its own rules, chosen at runtime.
 */
public class BankDemo {

    // One amount for every account, so any difference in outcome comes purely from the
account's own rules
    private static final double WITHDRAWAL_AMOUNT = 250.00;
    private static final double INVALID_DEPOSIT_AMOUNT = -20.00;

    public static void main(String[] args) {
        List<Account> accounts = List.of(
                new SavingsAccount("SAV-001", "Tendai Moyo", 1000.00),
                new CurrentAccount("CUR-001", "Rudo Chikwanha", 100.00, 500.00),
                new SavingsAccount("SAV-002", "Farai Ncube", 200.00),
                new CurrentAccount("CUR-002", "Nyasha Dube", 50.00, 100.00)
        );

        System.out.println("=== Harare Community Bank - Month-End Run (USD) ===");
```

```
        System.out.println();

        System.out.println("Input validation check:");
        accounts.get(0).deposit(INVALID_DEPOSIT_AMOUNT);
        System.out.println();

        for (Account account : accounts) {
            System.out.printf("--- %s | %s | %s ---%n",
                    account.getAccountNumber(), account.getAccountType(),
account.getAccountHolder());

            System.out.printf("  Balance before withdrawal: %s%n",
Account.formatUsd(account.getBalance()));
            account.withdraw(WITHDRAWAL_AMOUNT);
            System.out.printf("  Balance after withdrawal:  %s%n",
Account.formatUsd(account.getBalance()));

            System.out.printf("  Balance before month-end:  %s%n",
Account.formatUsd(account.getBalance()));
            account.endOfMonth();
            System.out.printf("  Balance after month-end:   %s%n",
Account.formatUsd(account.getBalance()));
            System.out.println();
        }

        System.out.println("=== Closing balances ===");
        for (Account account : accounts) {
            System.out.printf("  %s  %-15s %12s%n",
                    account.getAccountNumber(), account.getAccountHolder(),
Account.formatUsd(account.getBalance()));
        }
    }
}
```

OUTPUT



Task 5 Edge cases and sample output

- **Rejected savings withdrawal:** SAV-002 (Farai Ncube) has $200.00. Withdrawing $250 would leave -$50.00, below the $50.00 minimum, so the withdrawal is rejected and the balance stays at $200.00.
- **Overdraft within limit:** CUR-001 (Rudo Chikwanha) has $100.00. Withdrawing $250 leaves -$150.00, which is within the $500.00 limit, so the withdrawal is allowed.
- **Additional cases:** CUR-002 is blocked when a withdrawal would exceed its $100 overdraft limit, and a deposit of -$20 is rejected by the input validation.

output



